

Подходы к генерации изображений – GAN, Diffusion

Генерация картинок с помощью text2image моделей

Занятие 4

План занятия

Теоретическая часть:

- GAN
- VAE+GAN, VQ-VAE+GAN
- Diffusion модель

Генеративно-сопоставительные сети (GAN)

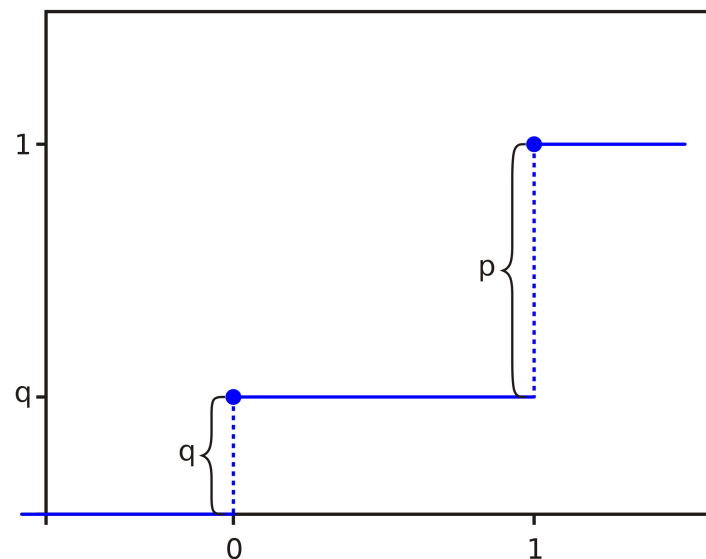


GAN

- Генерация случайных величин

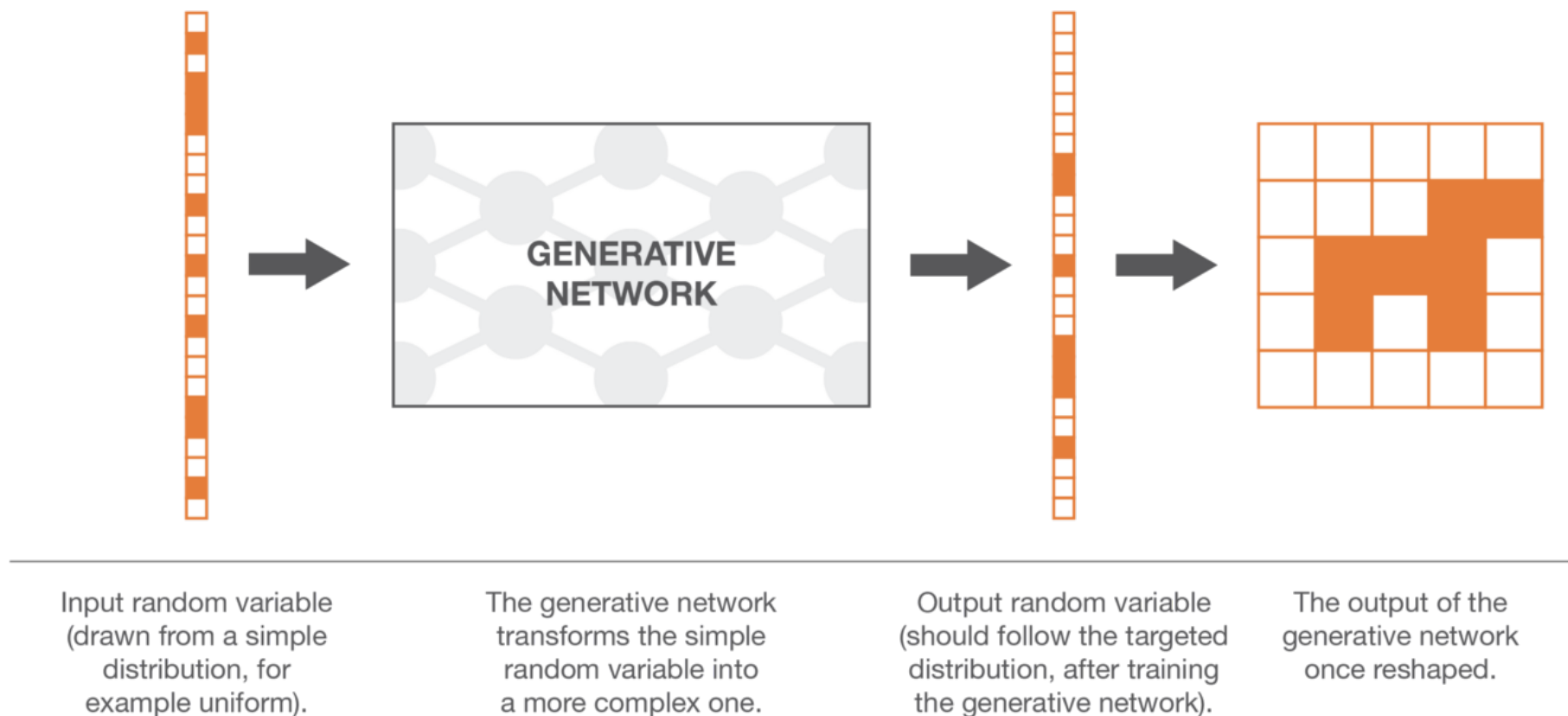
Случайная величина - переменная, значения которой представляют собой численные исходы некоторого случайного эксперимента.

Распределение вероятностей — это закон, описывающий область значений случайной величины и соответствующие вероятности появления этих значений.



GAN

- Генерация случайной величины из заданного распределения с помощью нейронной сети



GAN

X_S - реальные объекты (векторы изображений собак)

X_S лежат в каком-то сложном распределении $P(X)$, из которого мы хотим научиться генерировать новые объекты.

Генератор G

- берет случайные числа из какого-то заданного распределения $P(Z)$ (например, из $\mathcal{N}(0, 1)$)
- выдает сгенерированные объекты $X_g = G(x)$

Дискриминатор D

- получает на вход объекты из реальной выборки X_S и сгенерированной X_g
- выдает вероятность того, что объект реальный $D(x)$

GAN

Обучение

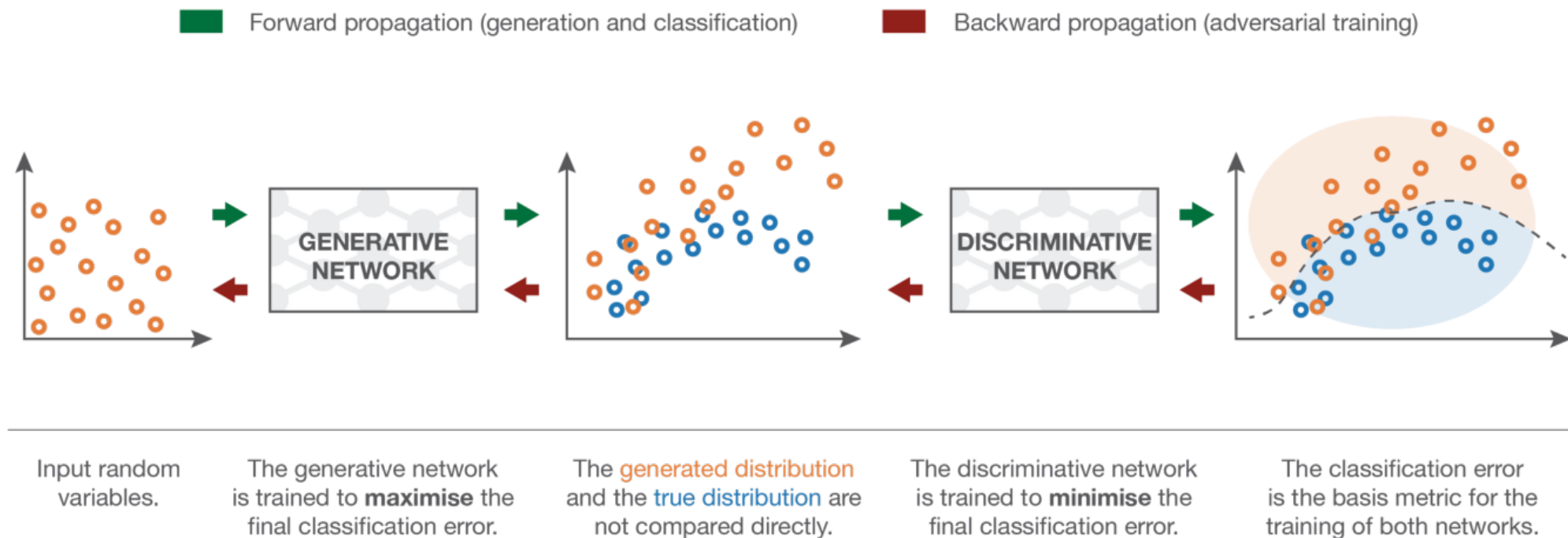
Генератор:

- Цель – обмануть дискриминатор
- Максимизирует ошибку классификации

Дискриминатор:

- Цель – обнаружить сгенерированные данные
- Минимизирует ошибку классификации

GAN

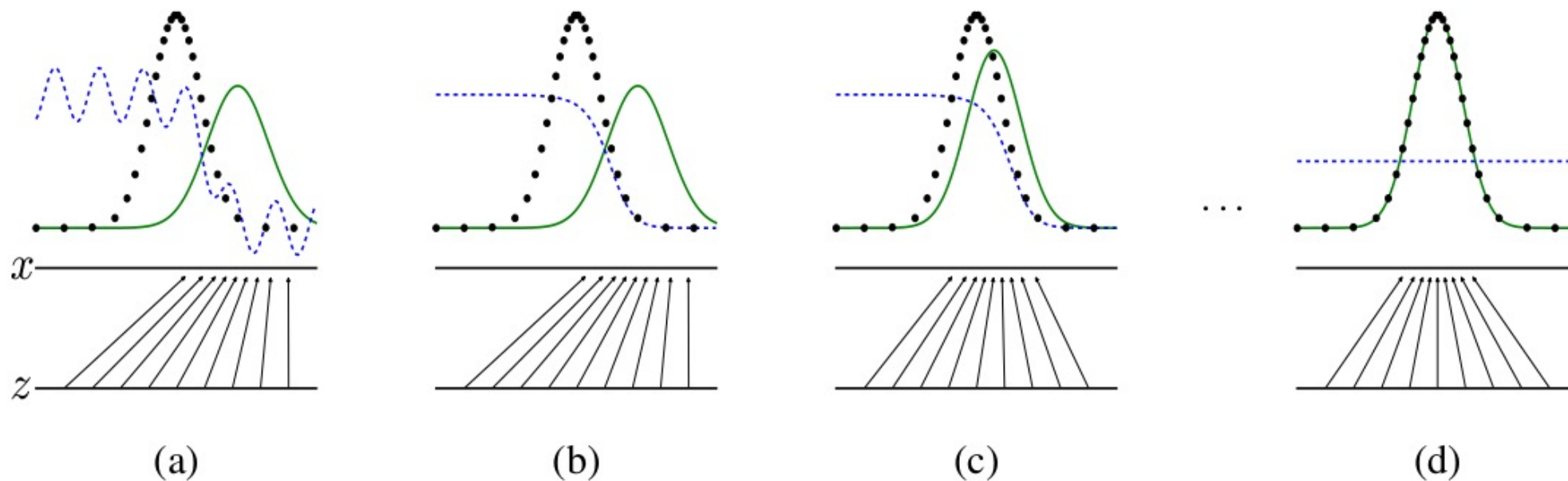


При заданном генераторе оптимальный дискриминатор выдает вероятность

$$D(X) = \frac{P(X)}{P_g(X) + P(X)}$$

GAN

При достаточной мощности обеих нейросетей у данной задачи есть оптимум, в котором генератор научился генерировать распределение $P_g(X)$, совпадающее с $P(X)$, а дискриминатор везде на X выдает вероятность $\frac{1}{2}$.



GAN

Генерация изображений - процесс обучения

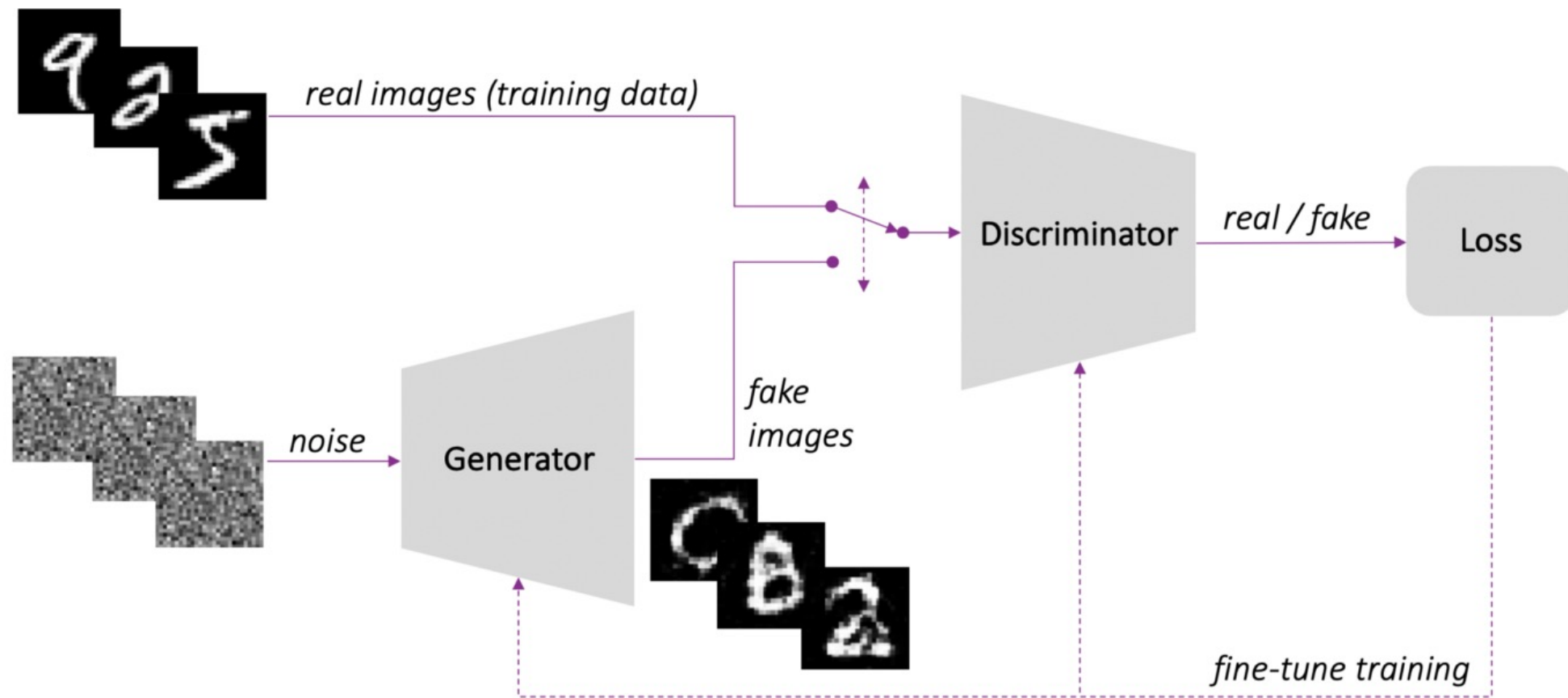
1. Обучаем дискриминатор:

- Замораживаем веса генератора
- Генерируем фейковые картинки с помощью генератора (вначале это просто шум, так как генератор еще не обучен)
- Берем выборку настоящих картинок
- Тренируем дискриминатор классифицировать картинки на настоящие и фейковые

2. Обучаем генератор:

- Замораживаем веса дискриминатора
- Генерируем фейковые картинки
- Тренируем генератор с помощью вывода дискриминатора на фейковых картинках

GAN



VAE + GAN = AAE

AAE – VAE, где для регуляризации вместо KL-дивергенции используется дискриминатор из GAN.

VAE: для приближения закодированных данных к нормальному распределению используем $D_{KL}(\mathcal{N}(\mu_x, \sigma_x), \mathcal{N}(0, 1))$

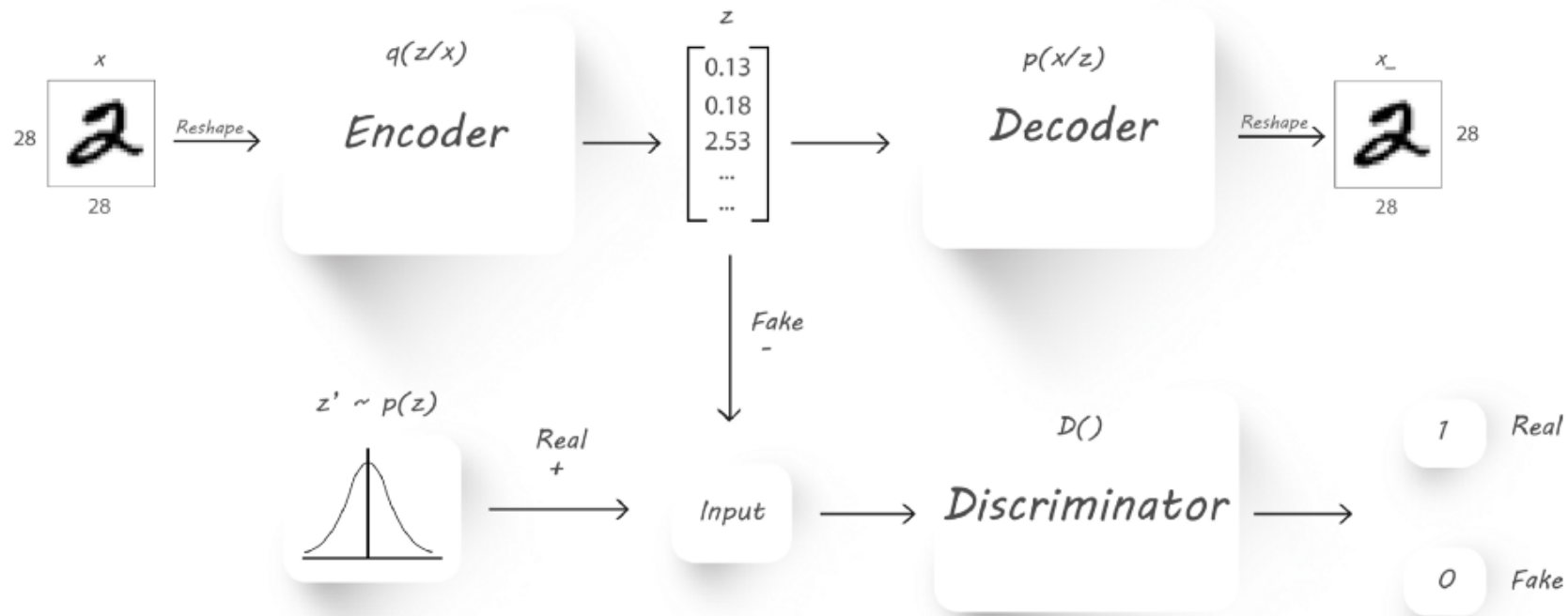
AAE: для приближения закодированных данных к нормальному распределению используем дискриминатор

GAN: выход генератора – изображение, дискриминатор отличает сгенерированные изображения от настоящих

AAE: в качестве генератора выступает энкодер VAE, его выход – распределение, задача дискриминатора – отличить вектор из настоящего нормального распределения от вектора из распределения энкодера

AAE

Cxema adversarial autoencoder:



ААЕ

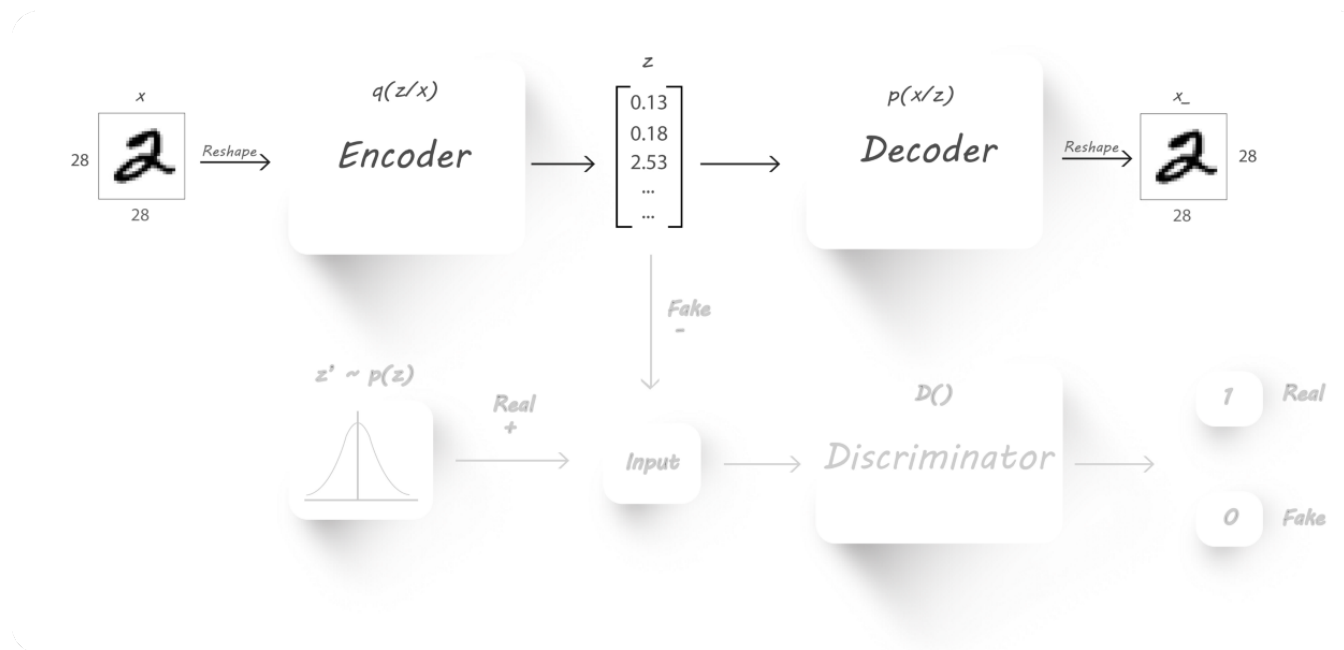
Компоненты:

- Энкодер
- Декодер
- Дискриминатор

AAE

Обучение – фаза реконструкции:

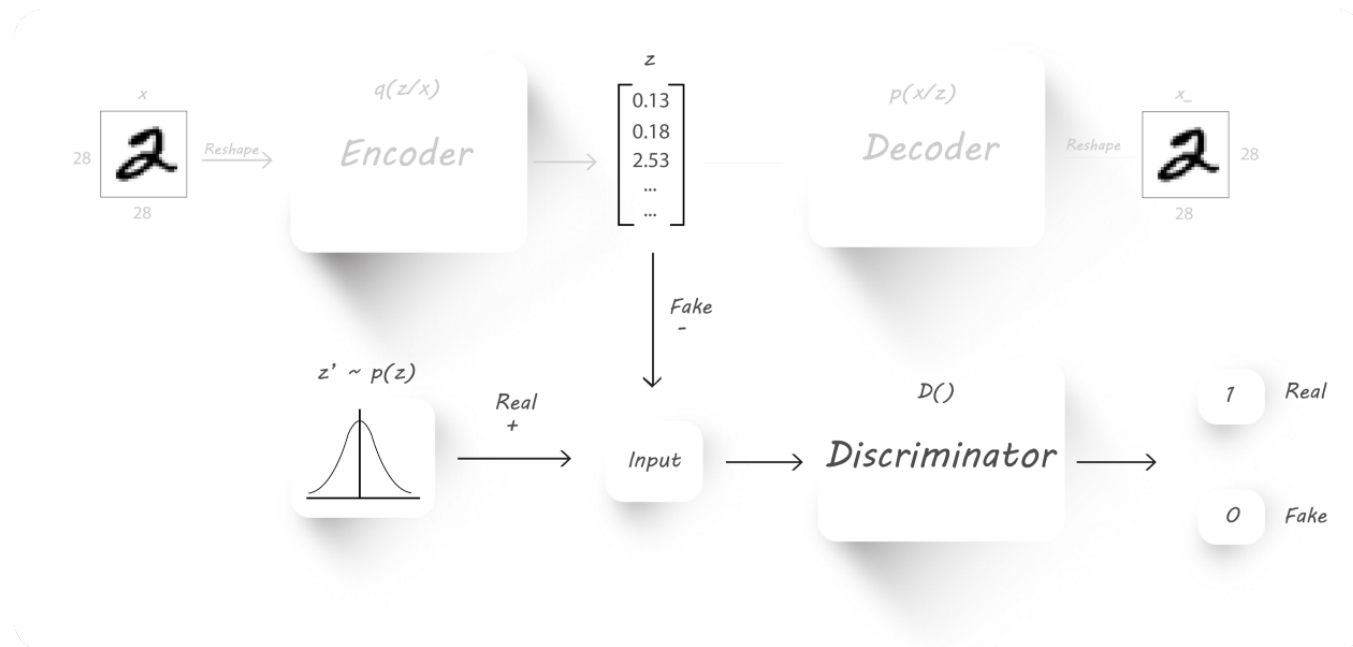
- Забываем про дискриминатор
- Обучаем VAE (энкодер + декодер) кодировать-декодировать изображения с наименьшими потерями



AAE

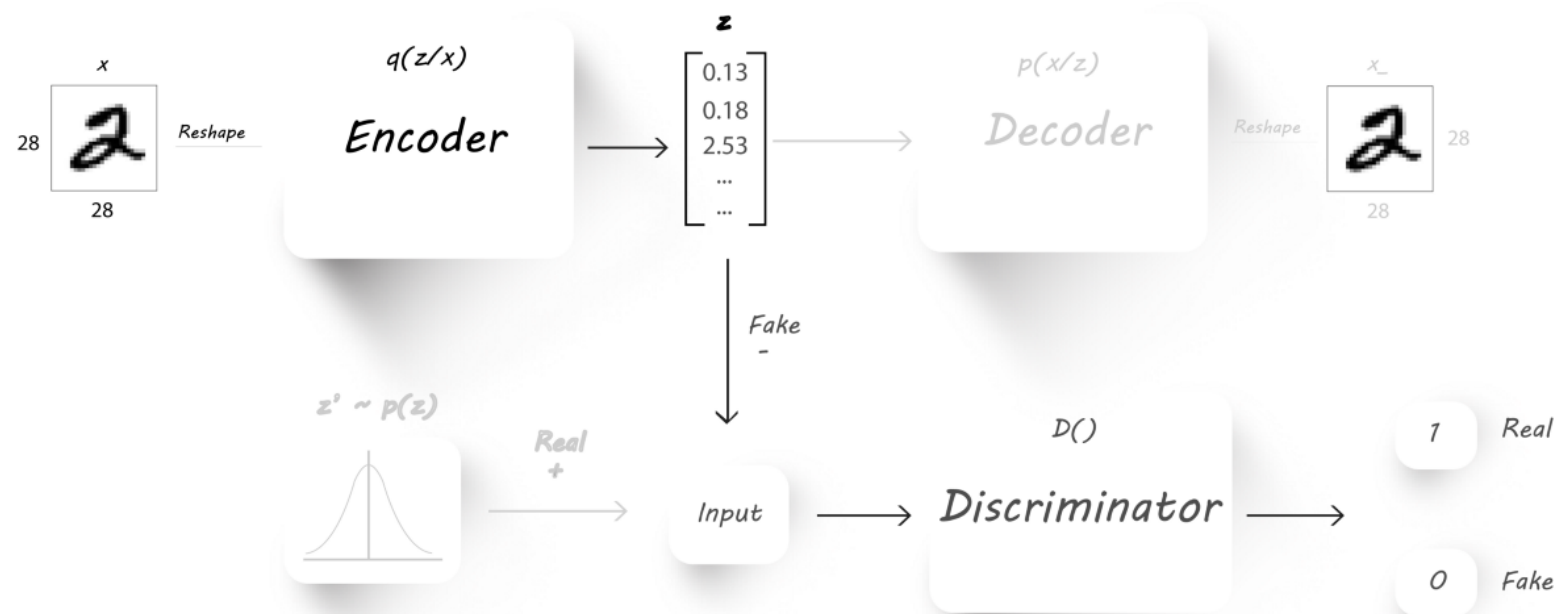
Обучение – фаза регуляризации:

- Забываем про декодер
- Обучаем связку энкодер + дискриминатор как GAN (сначала дискриминатор - классифицировать вектор из настоящего нормального распределения или из энкодера, потом энкодер – переводить изображение в нормальное распределение)



AAE

Обучение – фаза регуляризации:



VQ-VAE + GAN = VQGAN

VQ-GAN – VQ-VAE, где к reconstruction loss добавляется дискриминатор D .

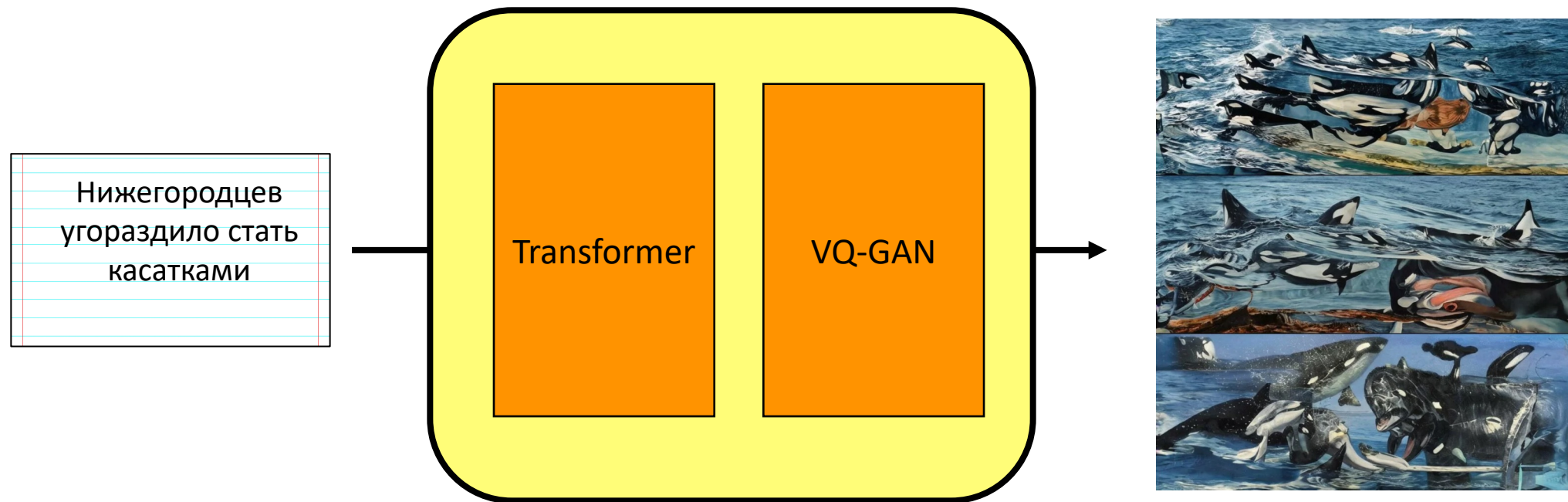
$$\text{VQ-VAE: } L_{VQ-VAE} = \|x - \hat{x}\|^2 + \|sg[z_e(x)] - e\|^2 + \beta \|z_e(x) - sg[e]\|^2$$

$$\text{VQ-GAN: } L = L_{VQ-VAE} + (\log D(x) + \log(1 - D(\hat{x})))$$

Общая схема ruDALL-E

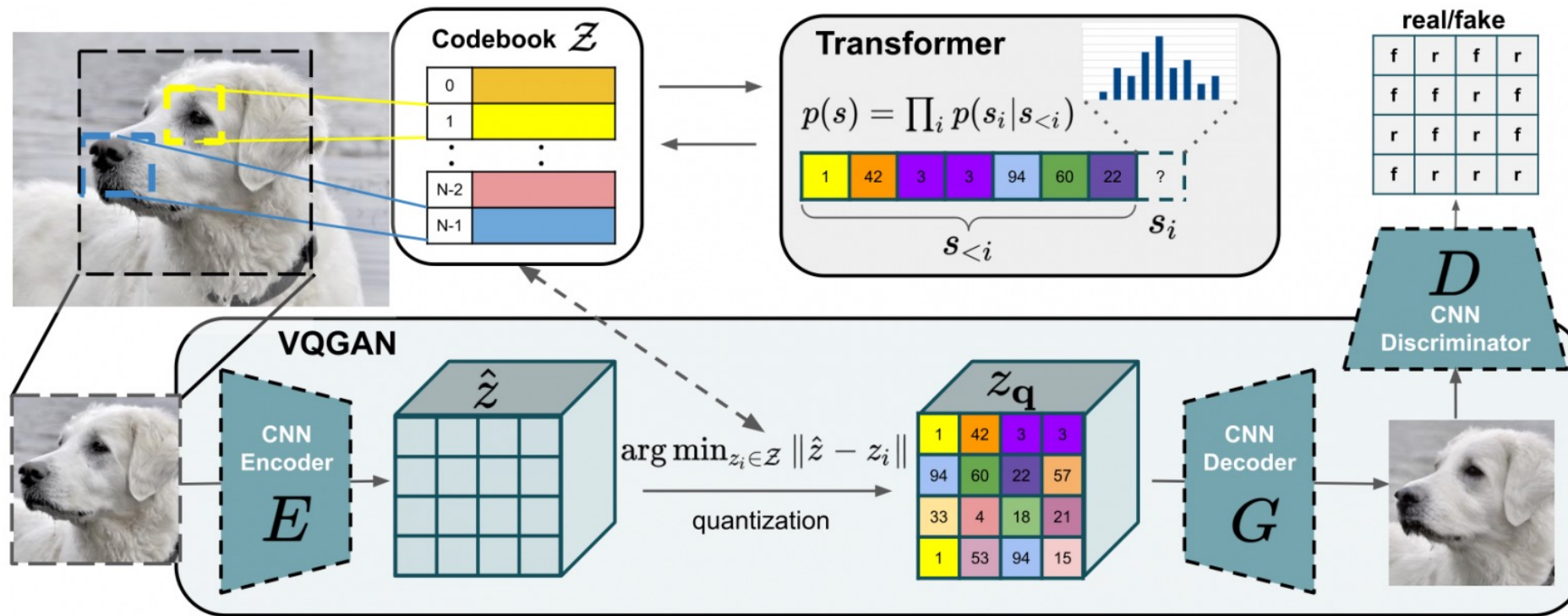
ruDALL-E, выпущены SberAI:

- ruDALL-E Malevich (1.3млрд параметров)
- ruDALL-E Kandinsky (12 млрд параметров)



ruDALL-E

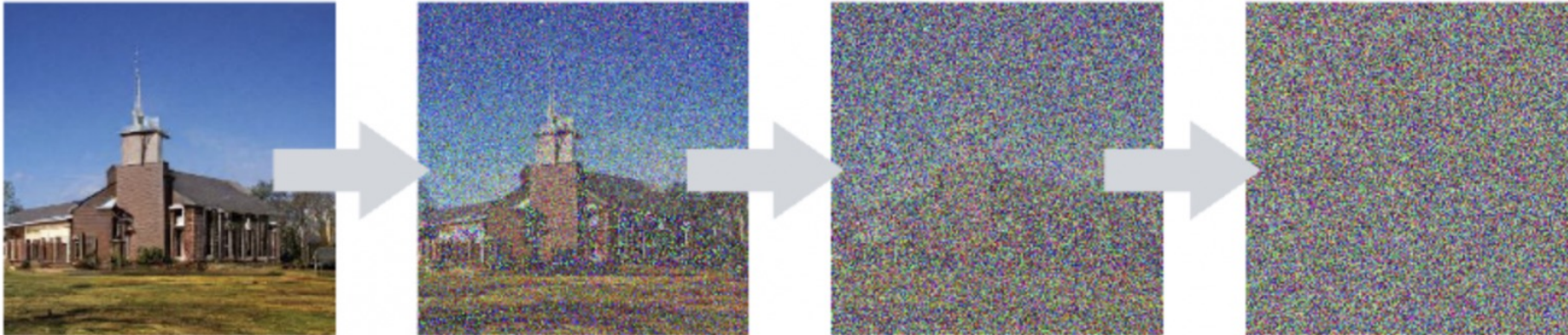
Схема архитектуры:



Diffusion модель

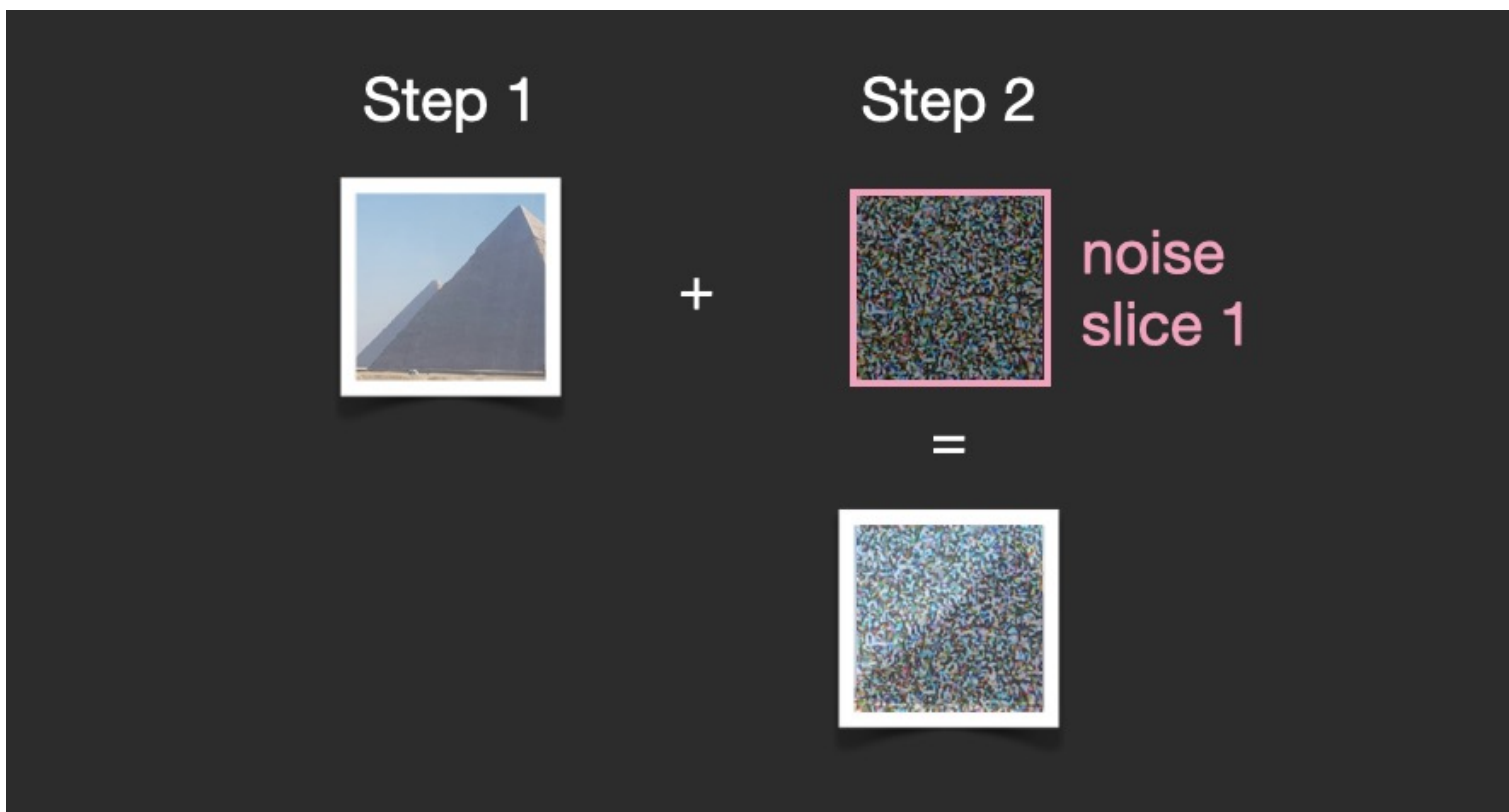
Идея

- Берем начальные изображения
- Итеративно добавляем Гауссовский шум, пока не получится каша из пикселей
- Обучаем модель восстанавливать начальные изображения из зашумленных



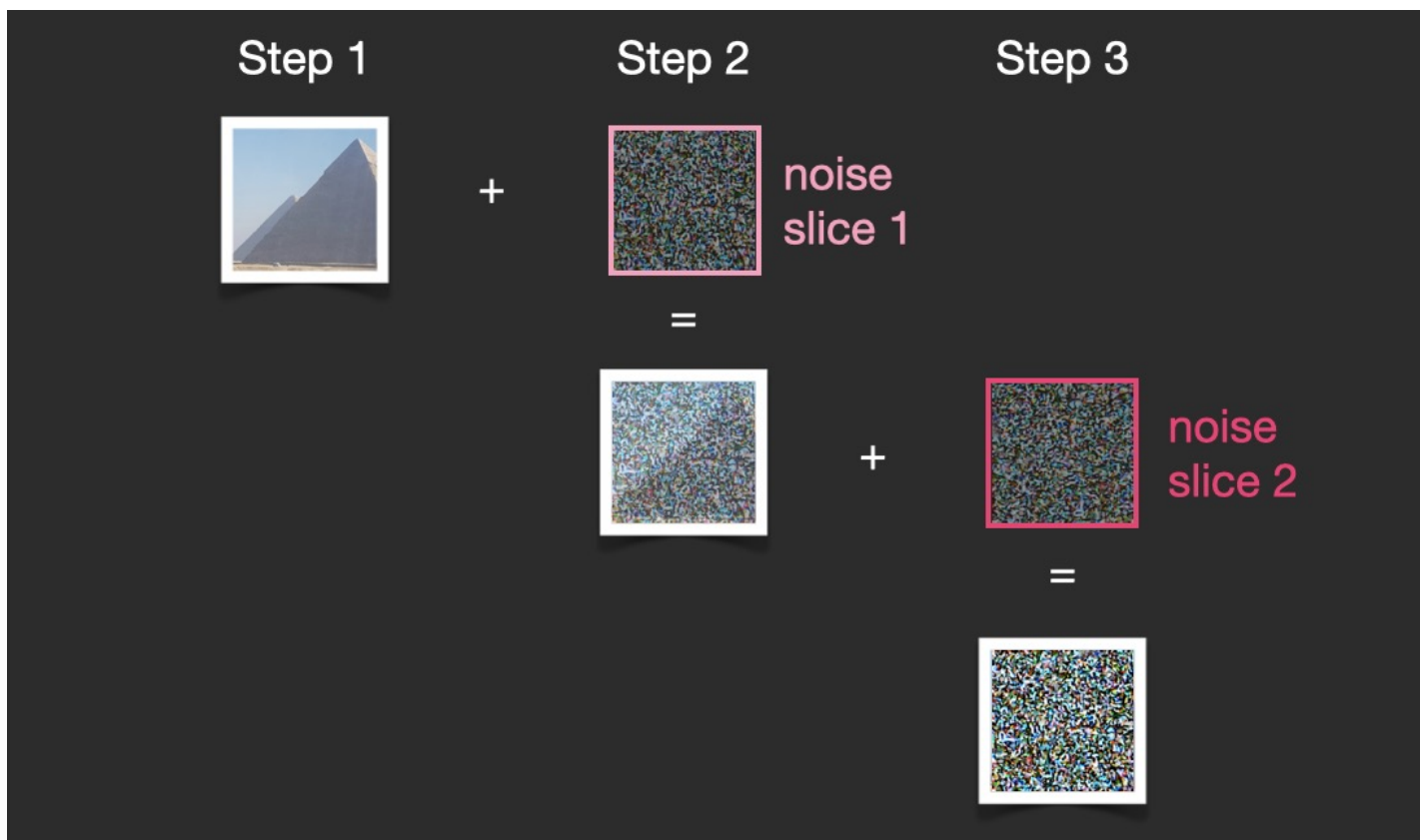
Diffusion модель

Построение датасета (зашумление – прямой процесс диффузии)



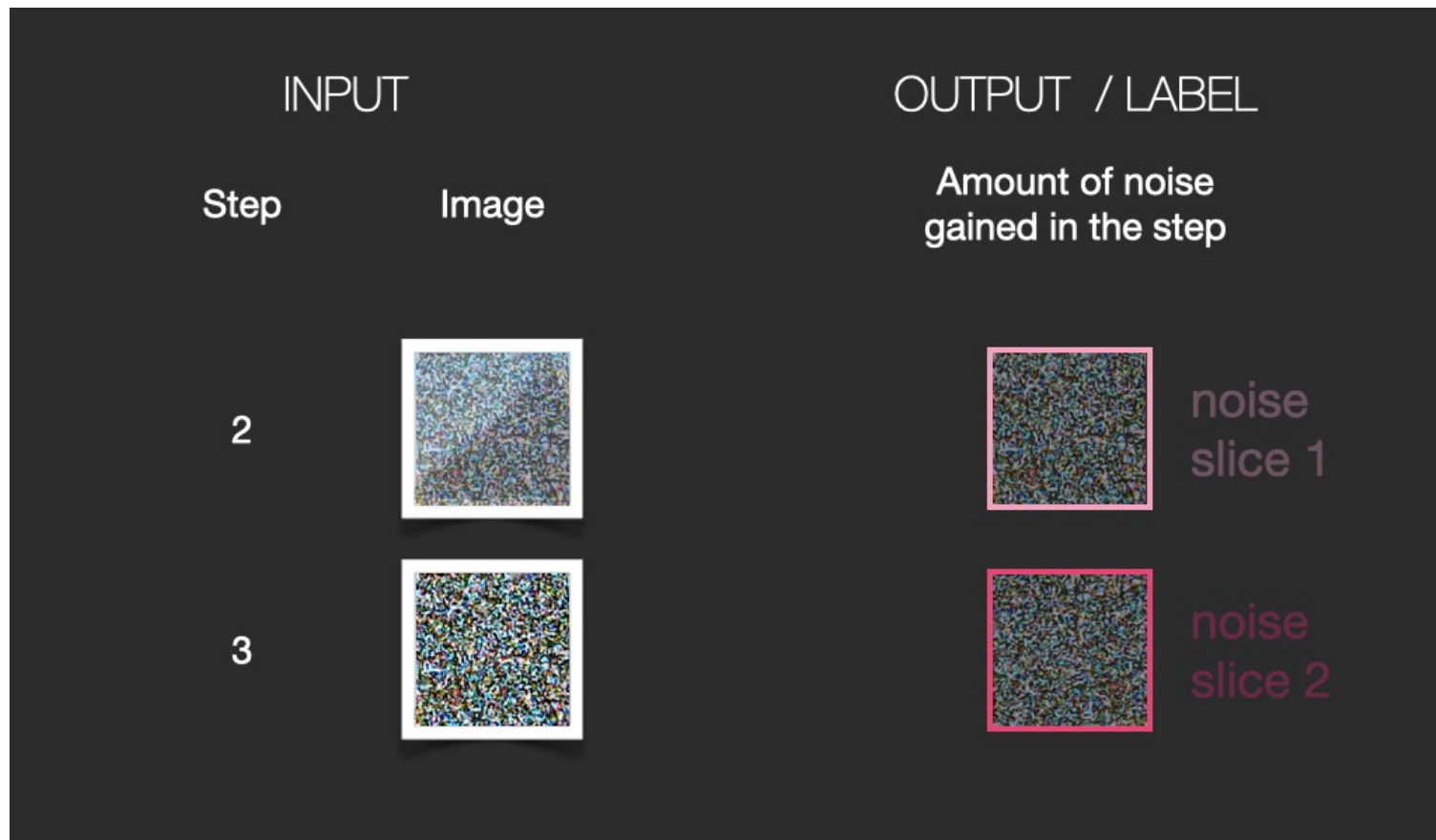
Diffusion модель

Построение датасета (зашумление – прямой процесс диффузии)



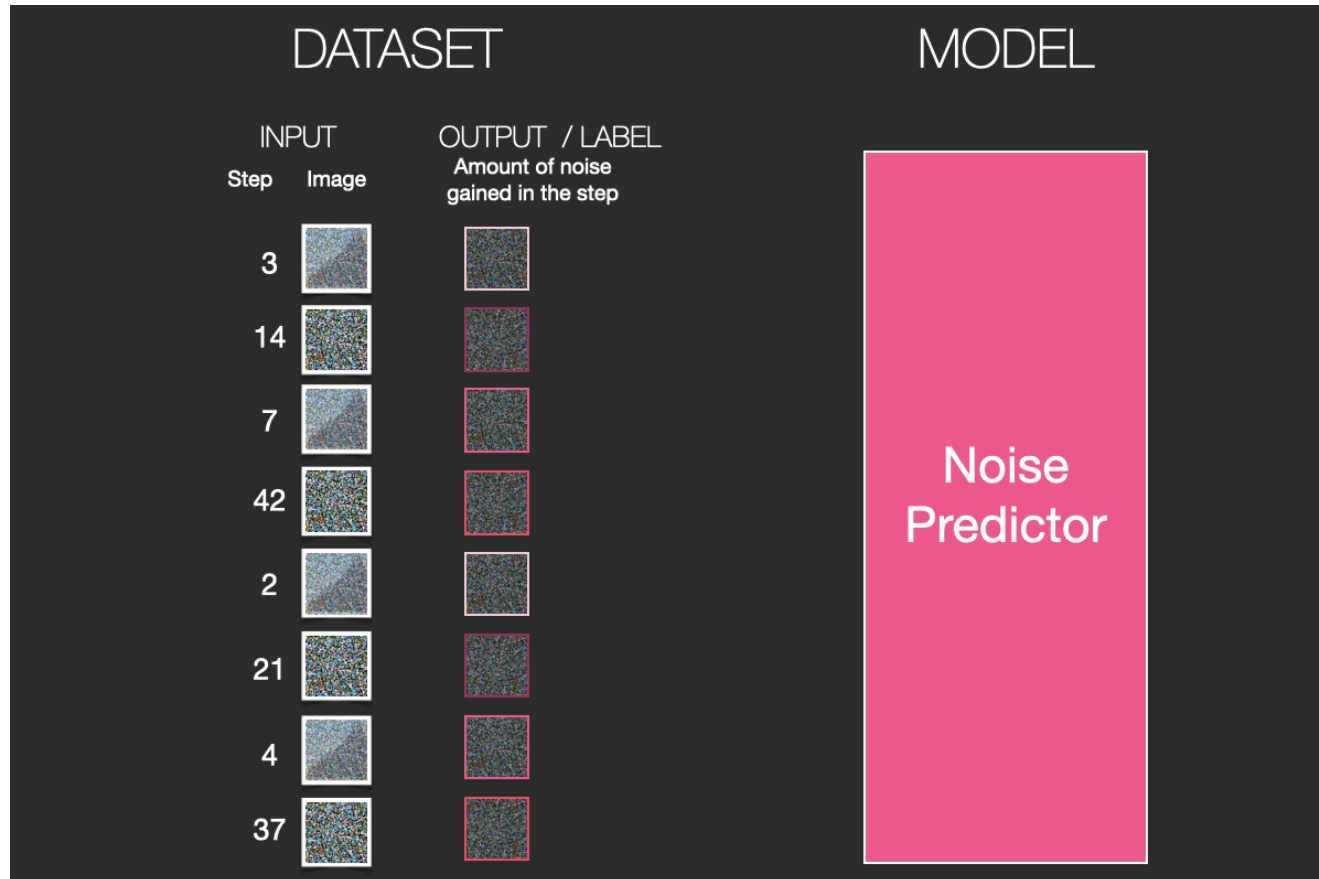
Diffusion модель

Построение датасета (зашумление – прямой процесс диффузии)



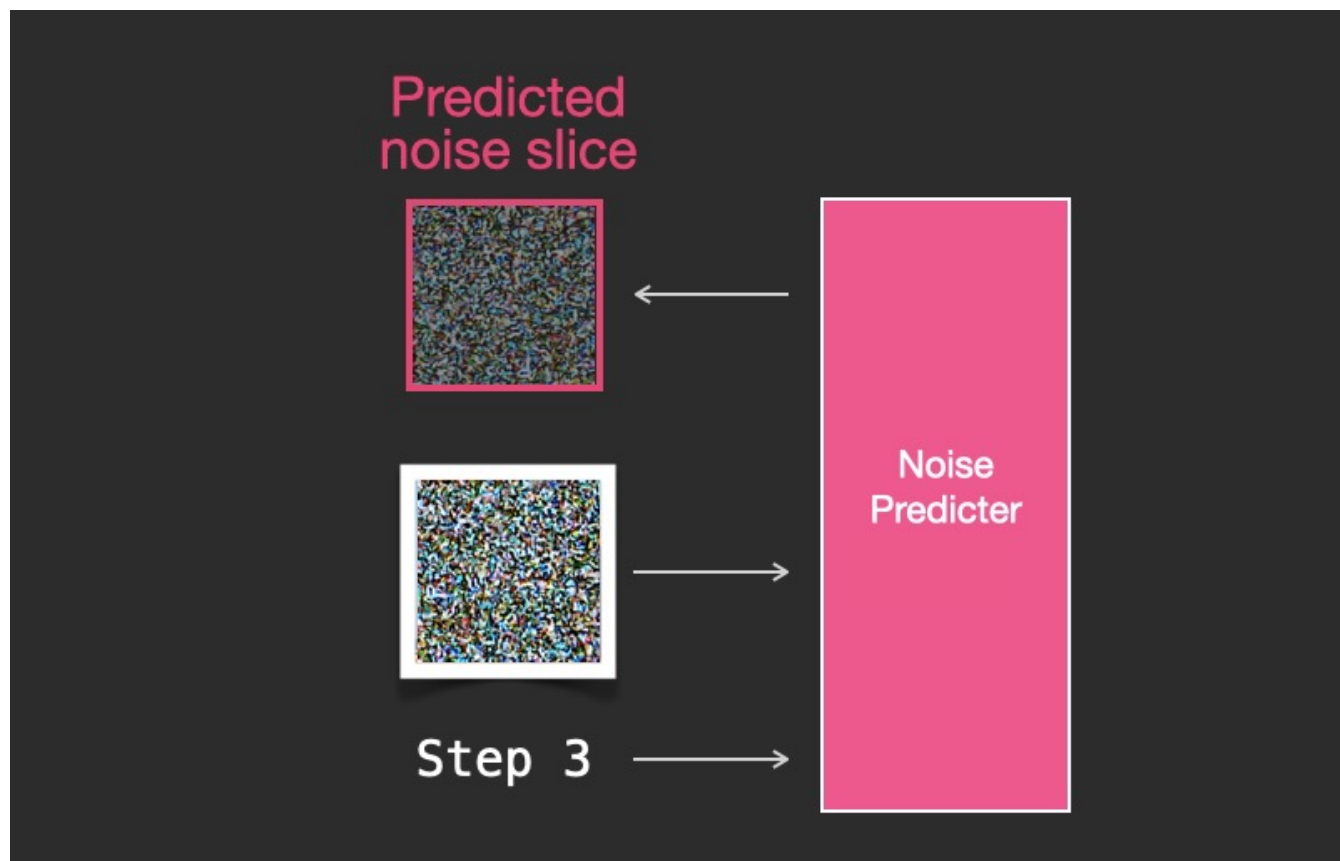
Diffusion модель

Построение датасета (зашумление – прямой процесс диффузии)



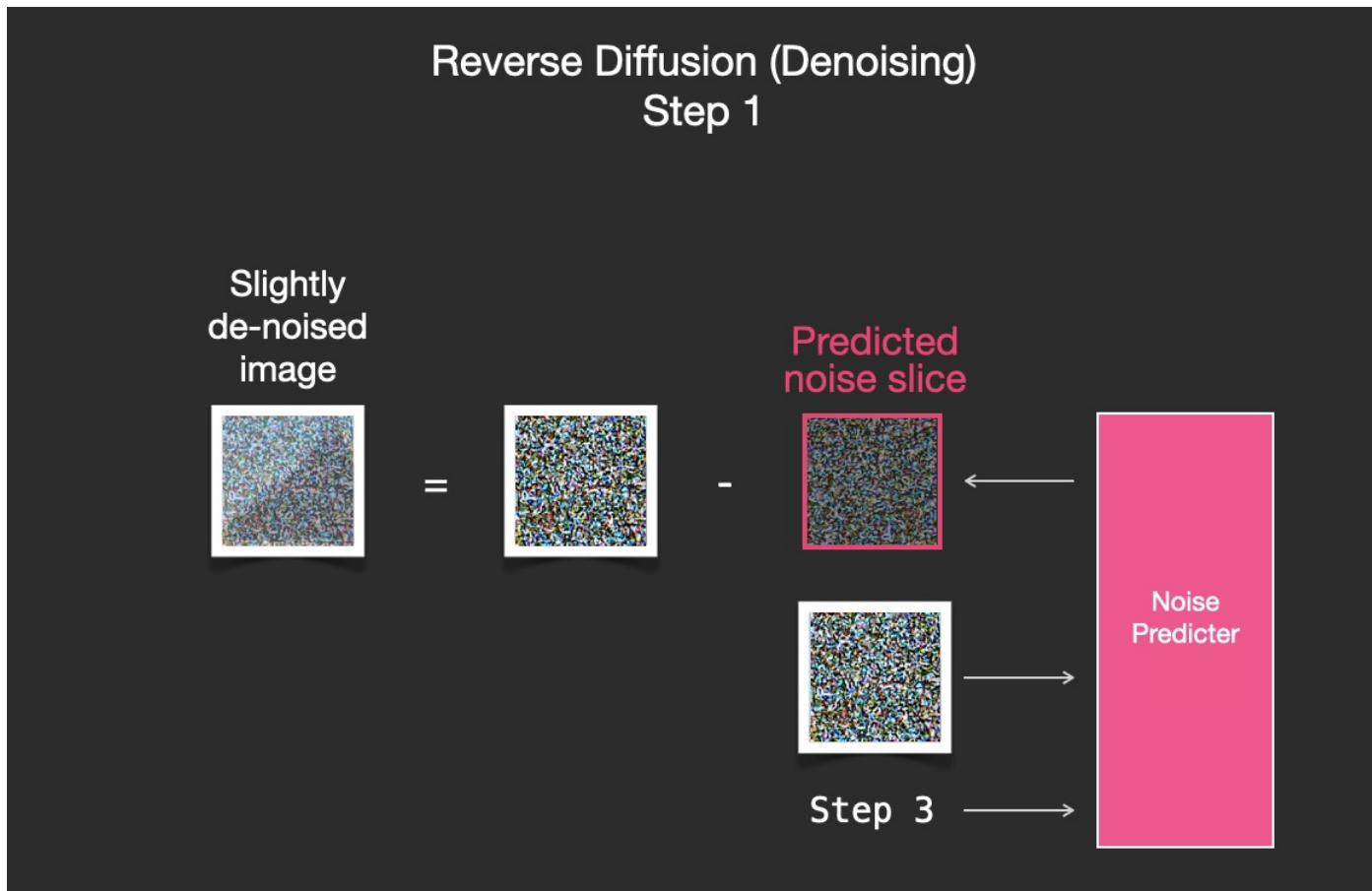
Diffusion модель

Восстановление изображения (обратная диффузия)



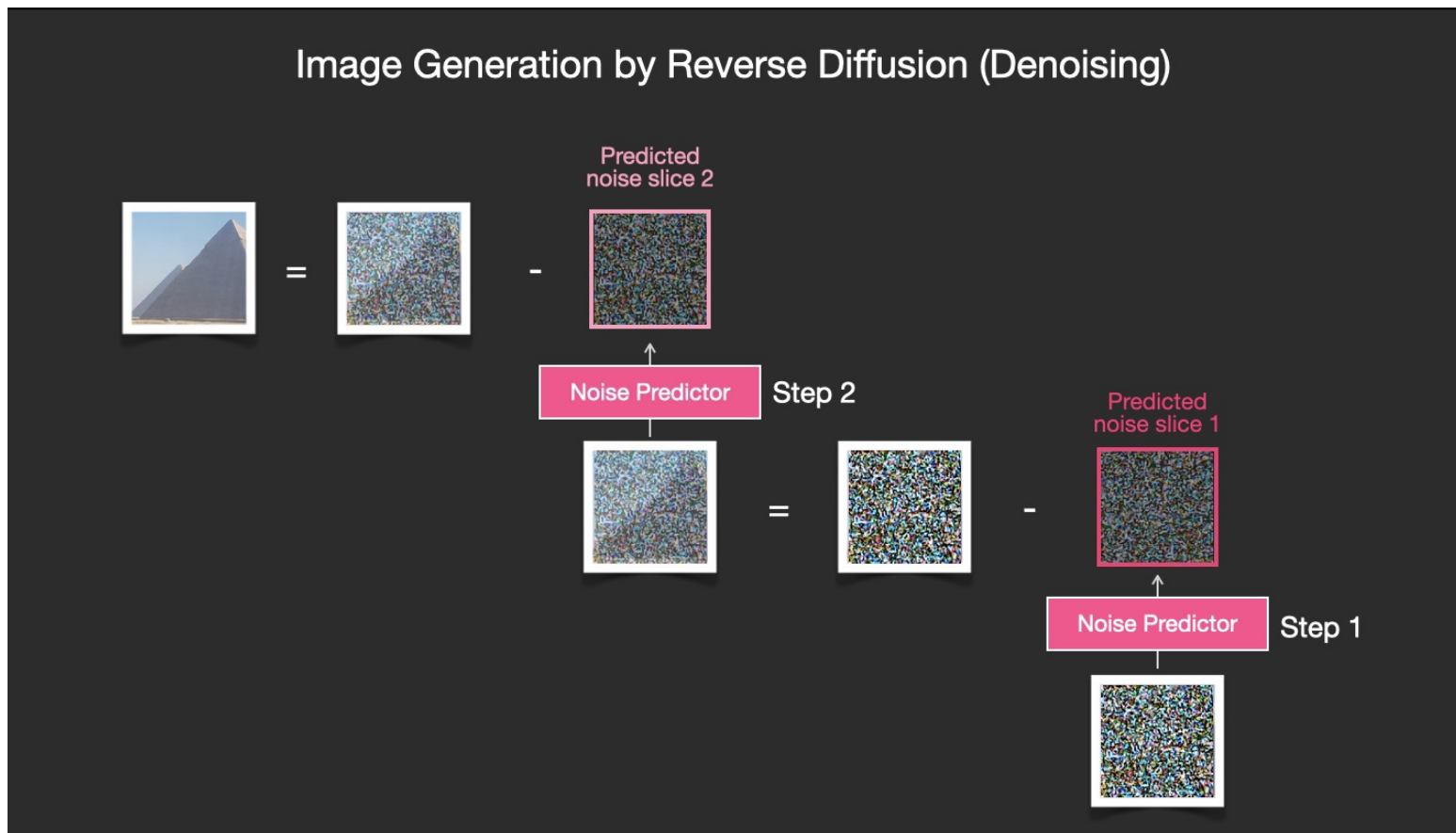
Diffusion модель

Восстановление изображения (обратная диффузия)



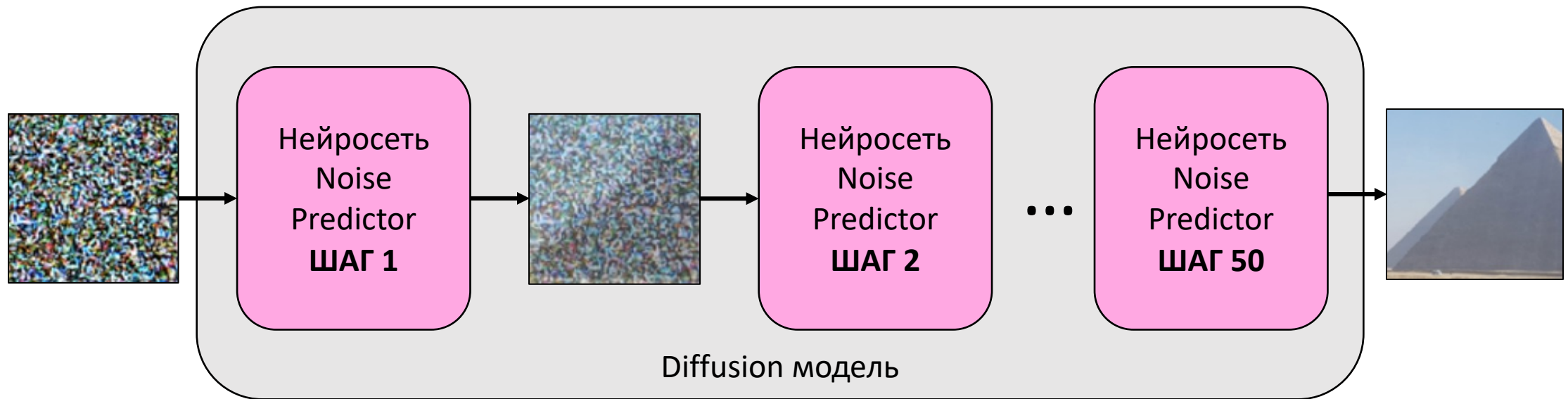
Diffusion модель

Восстановление изображения (обратная диффузия)



Diffusion модель

Восстановление изображения (обратная диффузия)



Статьи

- Подробное объяснение про GAN <https://towardsdatascience.com/understanding-generative-adversarial-networks-gans-cd6e4651a29>
- Кратко про GAN, GAN + VAE <https://medium.com/vitrox-publication/adversarial-auto-encoder-aae-a3fc86f71758>
- Подробно про AAE <https://towardsdatascience.com/a-wizards-guide-to-adversarial-autoencoders-part-2-exploring-latent-space-with-adversarial-2d53a6f8a4f9>
- Оригинальная статья про диффузию <https://arxiv.org/pdf/2006.11239.pdf>
- Как работает Stable Diffusion (есть часть именно про диффузию)
<https://habr.com/ru/post/693298/>